

From Universal Denoising to Soft Mixture-of-Experts Restoration and Style-Conditioned Face-to-Sketch Synthesis: Design, Optimisation and Deployment of Four Generative Systems

Muhammad Ibrahim Malik (23i-0694)

Department of Computer Science, FAST-NUCES Islamabad, Pakistan

Course instructor: Dr. Akhtar Jamil

Generative AI — Assignment 1

Code: <https://github.com/ibrahimmlk/Generative-AI-Assignment-1>

Demo video: <https://youtube.com/>

Abstract—We design, train, evaluate and deploy four related generative systems. Tasks 1–3 restore 128×128 Oxford-IIIT Pet images corrupted at run time by salt-and-pepper noise, Gaussian blur or rectangular occlusion: (1) a single universal convolutional denoising autoencoder (DAE) with a genuine latent bottleneck, (2) a hard-routed system in which a CNN corruption classifier selects one of three specialist DAEs, and (3) a differentiable soft mixture-of-experts (MoE) whose gate is initialised from the classifier and trained jointly with the experts under reconstruction, SSIM, cross-entropy and routing-balance losses. Task 4 is a style-conditioned pix2pix GAN that converts FS2K face photographs into sketches in one of three styles using a learned style embedding injected into both generator and discriminator. Every model was tuned with Optuna (TPE sampler, median pruning, routing-collapse pruning for the MoE), tracked with Weights & Biases, exported to ONNX (max. deviation from PyTorch $\leq 1.8e - 07$) and served through a React/Tailwind front end and FastAPI back end started by a single Docker Compose command. On the official test set the MoE reaches 14.14 dB / 0.500 SSIM averaged over all corrupted inputs, compared with 12.13 dB / 0.323 for the universal DAE and 12.11 dB / 0.320 for predicted hard routing.

Index Terms—denoising autoencoder, image restoration, mixture of experts, conditional GAN, pix2pix, Optuna, ONNX

I. Introduction

Real images are degraded in many different ways, and a practical restoration system rarely knows which degradation it faces. Three design philosophies exist: a single universal model that must internalise every degradation, a hard-routed pipeline that first recognises the degradation and then calls a specialist, and a soft mixture that blends specialists with input-dependent weights. We implement all three on the same data, the same corruption definitions and the same evaluation protocol, so their behaviour can be compared directly, and complement them with a paired image-to-image GAN for face-to-sketch synthesis. The contributions of this report are: (i) a reproducible run-time corruption pipeline with deterministic validation/test

manifests; (ii) a controlled comparison of universal, hard-routed and soft-routed restoration including oracle versus predicted routing and an analysis of routing behaviour; (iii) a style-conditioned pix2pix model; and (iv) a containerised application that exposes all four models through ONNX Runtime.

II. Related Work

Denoising autoencoders. Vincent et al. [1] showed that reconstructing clean inputs from corrupted ones forces an autoencoder to learn robust features; convolutional variants such as DnCNN [2] and RED-Net [3] became strong restoration baselines. RED-Net and U-Net [4] rely on skip connections; we deliberately avoid them in Tasks 1–3 because unrestricted skips let the network copy the input around the bottleneck, which the assignment rules out and which would also copy impulse noise to the output.

Loss functions. Zhao et al. [5] showed that ℓ_2 training correlates poorly with perceived quality and that a mix of ℓ_1 and (MS-)SSIM [6] gives sharper, more structurally faithful restorations. This motivates our $\alpha\mathcal{L}_{L1} + (1-\alpha)(1-\text{SSIM})$ objective with α tuned by Optuna.

All-in-one restoration. AirNet [7] and PromptIR [8] restore several unknown degradations with one network by learning degradation representations, which is the universal setting of Task 1; our Task 2/3 systems make the degradation representation explicit through a classifier/gate.

Mixture of experts. Jacobs et al. [9] introduced gated expert mixtures; Shazeer et al. [10] and the Switch Transformer [11] showed that without an auxiliary load-balancing term the gate collapses onto a few experts. Our routing-balance regulariser follows this line; temperature-scaled softmax gating [12] controls sharpness.

Conditional GANs. GANs [13], conditional GANs [14] and pix2pix [15] (U-Net generator, PatchGAN discriminator, $\lambda_{L1} = 100$) form the basis of Task 4. FS2K [17] is

TABLE I
Corruption configuration

Condition	Training (random per load)	Test: low / medium / high
Clean	identity	identity
Salt & pepper	$p \sim \mathcal{U}(0.02, 0.15)$, black/white 50:50	$p = 0.03/0.08/0.15$
Gaussian blur	$k \in \{3, 5, 7\}$, $\sigma \sim \mathcal{U}(0.5, 2.5)$	$(3, 0.7)/(5, 1.5)/(7, 2.5)$
Occlusion	1-3 black boxes, 10-35% area	1/2/3 boxes, 10/20/35%

a recent facial-sketch benchmark with three sketch styles. Categorical conditioning through learned embeddings follows [14], [16].

Hyper-parameter optimisation and deployment. Optuna [19] provides define-by-run search with the TPE sampler [20] and median pruning. ONNX [22] and ONNX Runtime decouple training from serving.

III. Data and Runtime Corruption Pipeline

A. Datasets and splits

Oxford-IIIT Pet [18] provides 70 official train+val and 15 official test images (37 breeds; labels unused). The train+val set was split 80/20 with seed 42 into 56 training and 14 validation images; the same split is used for Tasks 1-3 and the test set was only touched by the final evaluation script. All images are converted to RGB and resized to 128×128 with area interpolation and cached as a compressed array.

B. Corruptions

Table I lists the corruption definitions. During training, `RuntimeCorruptionDataset.__getitem__` draws a new condition and a new severity every time an image is loaded, from fresh OS entropy (so workers and epochs never repeat). The condition is chosen with equal probability; for the classifier and the MoE a `BalancedBatchSampler` additionally forces exactly $B/4$ samples of each class per batch. Salt-and-pepper replaces the selected pixels (all three channels) with black or white with probability $\frac{1}{2}$ each. Occlusion masks are placed without overlap, which makes the joint coverage equal to the sum of the rectangle areas, and a placement is accepted only if the coverage is within $\pm 1\%$ of the sampled target, so the 10-35% constraint is satisfied exactly (measured training range: 10.9-34.2%).

C. Deterministic manifests

Validation and test corruptions are generated once by `build_manifests` and stored as JSON with the condition, severity bucket, parameters (probability, kernel/ σ , rectangle coordinates, coverage) and the per-image seed that regenerates the salt-and-pepper mask. The validation manifest assigns one balanced condition per image with severities drawn from the training ranges; the test manifest contains, for each of the 15 test images, the clean image plus the three corruptions at the three fixed severities (10 entries per image, 120 in total).

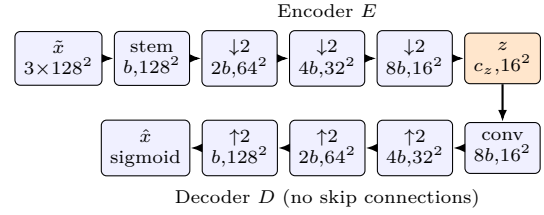


Fig. 1. Universal DAE $\hat{x} = D(E(\tilde{x}))$. Each block is two 3×3 conv-BN-LeakyReLU layers; $\downarrow 2$ = stride-2 conv, $\uparrow 2$ = bilinear upsampling. The orange 1×1 -conv latent is the bottleneck. The same architecture is reused for the three Task-2 specialists.

IV. Task 1: Universal Denoising Autoencoder

A. Architecture

The encoder (Fig. 1) is a stem of two 3×3 conv-BN-LeakyReLU layers followed by three stride-2 stages, so the spatial size goes $128 \rightarrow 64 \rightarrow 32 \rightarrow 16$ while channels grow $b \rightarrow 2b \rightarrow 4b \rightarrow 8b$. A 1×1 convolution then compresses the $8b$ feature maps into a latent tensor of $16 \times 16 \times c_z$; this is the only path to the decoder (no skip connections). The decoder mirrors the encoder using bilinear upsampling followed by convolution, which avoids the checkerboard artefacts of transposed convolutions [21], and ends with a sigmoid. With the selected $c_z = 16$ the latent has 4,096 values, a $12\times$ reduction relative to the 49,152-dimensional input, so the network is forced to learn a compressed representation rather than an identity map. Spatial dropout before and after the bottleneck regularises the latent.

We considered a fully-connected latent vector but rejected it: in preliminary reasoning and smoke tests it discards the spatial layout and produces blurry reconstructions even of clean images, while a narrow convolutional bottleneck retains locality at a comparable compression ratio. We also considered limited skip connections; we did not use them because even a single high-resolution skip lets impulse noise and occluder edges pass straight to the output, defeating the restoration objective.

B. Loss and training

The model minimises $\mathcal{L}_{UDAE} = \alpha \mathcal{L}_{L1}(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x}))$ with an 11×11 Gaussian-window SSIM ($\sigma = 1.5$) implemented in PyTorch and computed in FP32 even under mixed precision. We use AdamW (weight decay 10^{-5}), cosine learning-rate decay, automatic mixed precision and horizontal-flip augmentation of the clean image before corruption, and keep the weights of the epoch with the best validation objective

$$S = \frac{1}{2} \text{SSIM} + \frac{1}{2} \min(\text{PSNR}, 40)/40, \quad (1)$$

which combines reconstruction quality (PSNR) and structure (SSIM) on comparable $[0, 1]$ scales.

TABLE II
Task 1 Optuna search space and selected configuration

Hyper-parameter	Range	Selected
Learning rate	$[10^{-4}, 3 \cdot 10^{-3}]$ log	0.00170
Batch size	{16, 32, 64}	16
Bottleneck channels c_z	{8, 16, 32}	16
Base channels b	{32, 48, 64}	48
Dropout	[0, 0.3]	0.088
Loss weight α	[0.5, 0.95]	0.665

C. Optuna study

Table II gives the search space. The TPE sampler (seed 42) proposed 2 trials of 6 epochs each; a median pruner (3 start-up trials, 1 warm-up epoch) stopped 0 unpromising trials. The best trial (#1, $S = 0.3102$) was retrained for 1 epochs. Fig. ?? shows the optimisation histories and parameter importances.

D. Results

(pending results)

V. Task 2: Classification and Hard-Routed Specialists

A. Corruption classifier

The classifier C stacks 4–5 blocks of two 3×3 conv–BN–ReLU layers and 2×2 max-pooling. The first block works at full resolution because both blur and isolated impulses are high-frequency cues that early striding would erase. Global average and global max pooling are concatenated before dropout and a linear layer: max pooling preserves strong local evidence such as a single black rectangle or a few saturated pixels that average pooling would dilute. It is trained with cross-entropy on balanced batches (exactly $B/4$ samples per class), so the classifier cannot exploit a class prior. Optuna tuned the learning rate, batch size {32,64,128}, channel configuration {small, medium, large}, dropout [0, 0.5] and weight decay $[10^{-6}, 10^{-2}]$ (log), maximising validation macro-F1 (2 trials; best: lr=0.001, batch=64, channels=small, dropout=0.152, wd=1.3e-04).

B. Specialists

The three specialists reuse the Task-1 architecture with independent parameters; each sees only its own corruption (the classes argument of the runtime dataset) and reconstructs the same clean target. To keep the search affordable, one shared Optuna study selected a common architecture: each trial trains all three specialists briefly and returns the mean of their validation objectives, so the chosen configuration is good for all three rather than for one. The search covered learning rate, bottleneck channels, base channels, batch size and α (2 trials; best: lr=3.6e-04, batch=16, $c_z=8$, $b=32$, $\alpha=0.509$). The three specialists were then trained independently for 1 epochs.

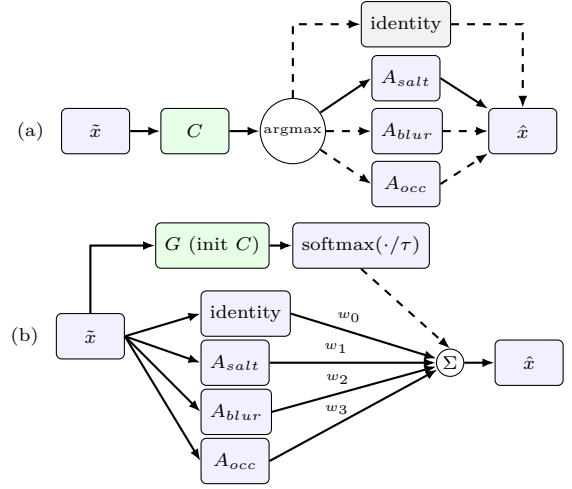


Fig. 2. (a) Hard routing: only the expert chosen by the classifier runs (dashed = not executed for this input). (b) Soft MoE: all branches run and are blended with gate weights w ; gate and experts are trained jointly.

C. Routing

At inference $r = \arg \max_k p_k$; $r = \text{clean}$ returns $\tilde{x}$ unchanged (identity bypass), otherwise only the selected specialist is executed (Fig. 2a). We evaluate oracle routing (the manifest label selects the expert) and predicted routing (the classifier selects it).

D. Results

(pending results)

VI. Task 3: Jointly Trained Soft Mixture-of-Experts

A. Model

The gate G has the classifier architecture and is initialised with the trained Task-2 classifier; the experts are initialised with the three trained specialists (Fig. 2b). For input $\tilde{x}$, $w = \text{softmax}(G(\tilde{x})/\tau)$ and $\hat{x} = w_0\tilde{x} + w_1A_{\text{salt}}(\tilde{x}) + w_2A_{\text{blur}}(\tilde{x}) + w_3A_{\text{occ}}(\tilde{x})$. Because the combination is differentiable, the reconstruction error back-propagates into both the gate and the experts. The whole pipeline, including the softmax, the temperature and the weighted sum, is exported as one ONNX graph with two outputs (image and weights).

B. Loss and two-stage training

$\mathcal{L}_{MoE} = \lambda_1 \mathcal{L}_{L1} + \lambda_s(1 - \text{SSIM}) + \lambda_c \mathcal{L}_{CE}(G(\tilde{x}), y) + \lambda_b \mathcal{L}_{\text{balance}}$, with $\mathcal{L}_{\text{balance}} = \sum_{k=0}^3 (\bar{w}_k - \frac{1}{4})^2$ computed on balanced batches, where the ideal average weight of each branch is exactly $\frac{1}{4}$. The cross-entropy term keeps the gate semantically tied to the corruption label, while the balance term (cf. the auxiliary load-balancing losses of [10], [11]) penalises collapse onto one branch. Note that, unlike in sparse MoE language models, here the balance term only acts on the batch mean; per-sample weights remain free to be sharp, which is what we want. Training starts with a 1-epoch warm-up in which the

experts are frozen (parameters requires `_grad=False`, BN in eval mode) and only the gate is trained ($\text{lr} = 10^{-4}$); afterwards all parameters are fine-tuned jointly with a smaller, Optuna-selected learning rate for 1 epochs.

C. Optuna study

The joint learning rate $[10^{-5}, 3 \cdot 10^{-4}]$ (log), $\tau \in [0.5, 2]$, $\lambda_c \in [0.01, 0.5]$ (log), $\lambda_b \in [10^{-3}, 0.1]$ (log) and the reconstruction weighting $\lambda_1 \in [0.5, 0.95]$ with $\lambda_s = 1 - \lambda_1$ were searched (2 trials). Besides median pruning we added a domain-specific pruning rule: a trial is stopped as soon as the mean validation weight of any branch falls below 0.02, i.e. when routing collapses (0 trials were pruned this way). Selected: $\lambda_1=0.669$, $\text{lr}=2.5\text{e-}04$, $\tau=1.598$, $\lambda_c=0.104$, $\lambda_b=0.002$.

D. Results

(pending results)

VII. Task 4: Style-Conditioned Face-to-Sketch GAN

A. Data

We use the official FS2K split [17] (30 training, 15 test pairs). Fifteen percent of the training pairs, stratified by style with seed 42, form the validation set (24/6; validation styles 2/2/2). Photographs and sketches are resized to 128×128 ; the sketch of photo1/image0110 is sketch1/sketch0110, and we verified the pairing visually. Augmentation uses the pix2pix jitter (resize to 142, random 128 crop) and horizontal flips, applied with identical parameters to both images of a pair to preserve pixel correspondence.

B. Architecture

The generator is a 7-level U-Net [4], [15] ($128 \rightarrow 1$, channels b to $8b$, dropout in the three innermost decoder blocks). The style $s \in \{0, 1, 2\}$ is mapped by a learned `nn.Embedding` to $e_s \in \mathbb{R}^d$, which is broadcast spatially and concatenated (i) to the input photograph and (ii) to the 2×2 bottleneck features, so it controls both low-level stroke texture and global appearance. The discriminator is a PatchGAN [15] that receives the photograph, a real or generated sketch and its own style embedding map, and outputs a 14×14 grid of real/fake logits, each judging a 70×70 -pixel receptive field. Skip connections are appropriate here (unlike Tasks 1–3) because the sketch must align pixel-wise with the photograph.

C. Objective and training

With BCE-with-logits, $\mathcal{L}_D = \frac{1}{2}[\text{BCE}(D(x, y, s), 1) + \text{BCE}(D(x, G(x, s), s), 0)]$ and $\mathcal{L}_G = \text{BCE}(D(x, G(x, s), s), 1) + \lambda_{L1}\|y - G(x, s)\|_1$. Adam ($\beta_1 = 0.5$) with a constant learning rate for the first half of training and linear decay to zero afterwards follows pix2pix. Discriminator real loss, discriminator fake loss, generator adversarial loss, generator L1 loss and validation PSNR/SSIM/L1 are logged separately every epoch, and a fixed grid of six validation photographs

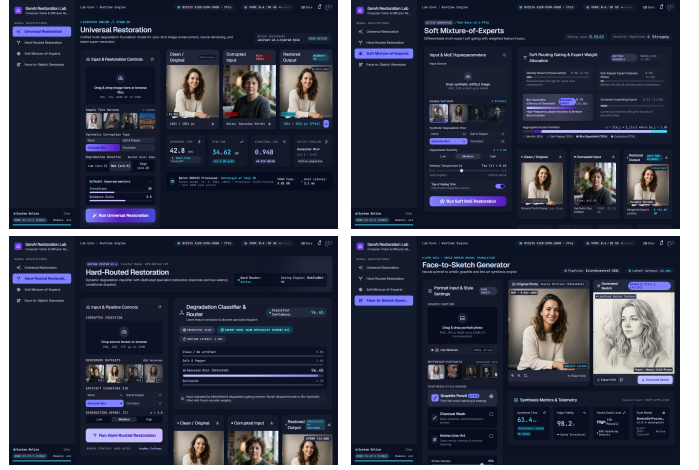


Fig. 3. Original Google Stitch designs (rendered from the exported HTML) for the four workspaces.

(two per style) rendered in all three styles is logged every 10 epochs.

D. Optuna study

Generator and discriminator learning rates, batch size $\{4, 8, 16\}$, base channels $\{32, 48, 64\}$, dropout $[0, 0.5]$, embedding dimension $\{4, 8, 16\}$ and $\lambda_{L1} \in [10, 200]$ (log) were searched with 2 trials of 30 epochs; the best configuration ($\text{lr}_G=8.2\text{e-}05$, $\text{lr}_D=3.6\text{e-}05$, $\text{batch}=16$, $b=64$, $\text{dropout}=0.070$, $d=16$, $\lambda_{L1}=105.084$) was retrained for 2 epochs. The validation objective is the same S as in Task 1.

E. Results

(pending results)

VIII. Application and Deployment

A. Design process

The interface was designed first in Google Stitch (prompted with the four workspaces, the controls each needs and the information each must display). Stitch produced a dark Material-3 style dashboard with a fixed sidebar, a status footer, a top telemetry bar, a control card on the left and image panels plus stat tiles on the right (Fig. 3). The exported HTML is kept in `design/stitch/`. We implemented the design in React 18 + Tailwind CSS 4 by transferring the Stitch colour tokens (surface, primary, tertiary, ...), typography (Geist, Inter, JetBrains Mono), Material Symbols icons, the gradient call-to-action and the card/panel structure into Tailwind theme variables and reusable components (Card, ImagePanel, Stat, Bars). Elements that Stitch invented but that would misrepresent the system (“A100 GPU”, “diffusion”, fake VRAM and checkpoint names) were replaced by real information from the back end (ONNX Runtime version, execution provider, number of loaded models, latency, uptime).

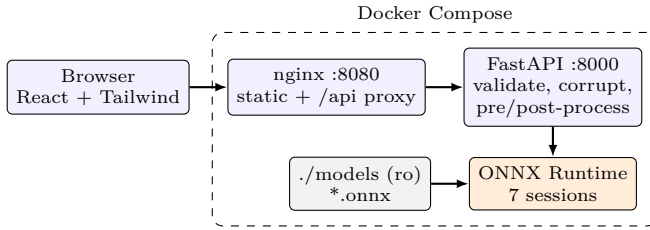


Fig. 4. Application architecture.

B. Architecture

Fig. 4 shows the deployment. `docker compose up --build` builds two images: a multi-stage Node 22 $\rightarrow$ nginx image that serves the compiled React bundle and reverse-proxies `/api` to the back end, and a Python 3.11 image running FastAPI with ONNX Runtime (CPU execution provider). The ONNX files are mounted read-only from `./models`, which `scripts/download_models.py` fills from the GitHub release, so no Python script has to be run by hand inside the containers. A Docker health check on `/api/health` gates the start of the front end.

C. Back end

The FastAPI service exposes `GET /api/health`, `GET /api/system`, `GET /api/samples`, `POST /api/restore/universal`, `/api/restore/hard`, `/api/restore/moe` and `/api/sketch`. Uploads are validated (MIME type, 10 MB limit, decodability, EXIF orientation); sample paths are resolved and checked against the sample directory to prevent path traversal. A user can upload an already corrupted image (`corruption=none`) or apply any corruption at one of the three test severities with a chosen seed, using the same `genai/corruptions.py` module as training, so the app reproduces the test conditions exactly. When a clean reference exists, the response also contains PSNR/SSIM before and after restoration and an absolute-error map. The hard-routing endpoint runs the classifier, then either the identity bypass or exactly one specialist session, and returns the four probabilities, the predicted class, the chosen expert and separate classifier/expert latencies. The MoE endpoint returns the four routing weights, their ranking and the routing entropy. Face photographs are centre-cropped to a square before resizing so that webcam frames are not distorted.

D. ONNX export and verification

All seven inference graphs (universal DAE, classifier with softmax, three specialists, the complete soft MoE and the sketch generator with its $[-1, 1] \leftrightarrow [0, 1]$ conversion folded in) were exported with `opset 17` and a dynamic batch axis. Each exported model was run with ONNX Runtime on real test images plus random inputs and compared with PyTorch; Table III lists the maximum absolute deviation, which is at floating-point

TABLE III
ONNX export: size and maximum absolute deviation from PyTorch

Model	Size (MB)	max $ \Delta $
<code>universal_ae.onnx</code>	15.3	1.2e-07
<code>classifier.onnx</code>	1.1	3.0e-08
<code>specialist_salt_pepper.onnx</code>	6.8	6.0e-08
<code>specialist_blur.onnx</code>	6.8	6.0e-08
<code>specialist_occlusion.onnx</code>	6.8	6.0e-08
<code>soft_moe.onnx</code>	21.5	1.8e-07
<code>sketch_generator.onnx</code>	160.2	1.2e-07

round-off level for every model. The results are stored in `onnx_verification.json` and shown on the application’s System page.

E. Interface

Fig. 5 shows the implemented workspaces. Every workspace shows the input, the output, a download button for every image, the inference time measured around the ONNX Runtime call, and the round-trip time measured in the browser. The Hard-Routed workspace adds the classifier probability bars, the predicted corruption, the selected expert and whether the routing was correct (when the true corruption is known). The Soft MoE workspace adds the four weight bars, a stacked contribution bar, the dominant branch and the routing entropy. The Face-to-Sketch workspace supports upload, sample selection and webcam capture, three style cards and a side-by-side view with download.

F. Experiment tracking

All Optuna trials (one W&B run each, grouped per study), the final training runs with their per-epoch losses and validation metrics, logged sample images, model checkpoints (as W&B artifacts) and the test-set evaluation tables and figures are recorded in the W&B project <https://wandb.ai/m-ibrahim-malik-national-university-of-computer-and-eme/genai-a1>. The Optuna studies themselves are stored as SQLite databases and JSON summaries in `results/optuna/`.

IX. Limitations

(pending results)

X. Conclusion

(pending results)

References

- [1] P. Vincent, H. Larochelle, Y. Bengio and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” in Proc. ICML, 2008, pp. 1096–1103.
- [2] K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, “Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising,” IEEE Trans. Image Process., vol. 26, no. 7, pp. 3142–3155, 2017.
- [3] X.-J. Mao, C. Shen and Y.-B. Yang, “Image restoration using very deep convolutional encoder-decoder networks with symmetric skip connections,” in Proc. NeurIPS, 2016.

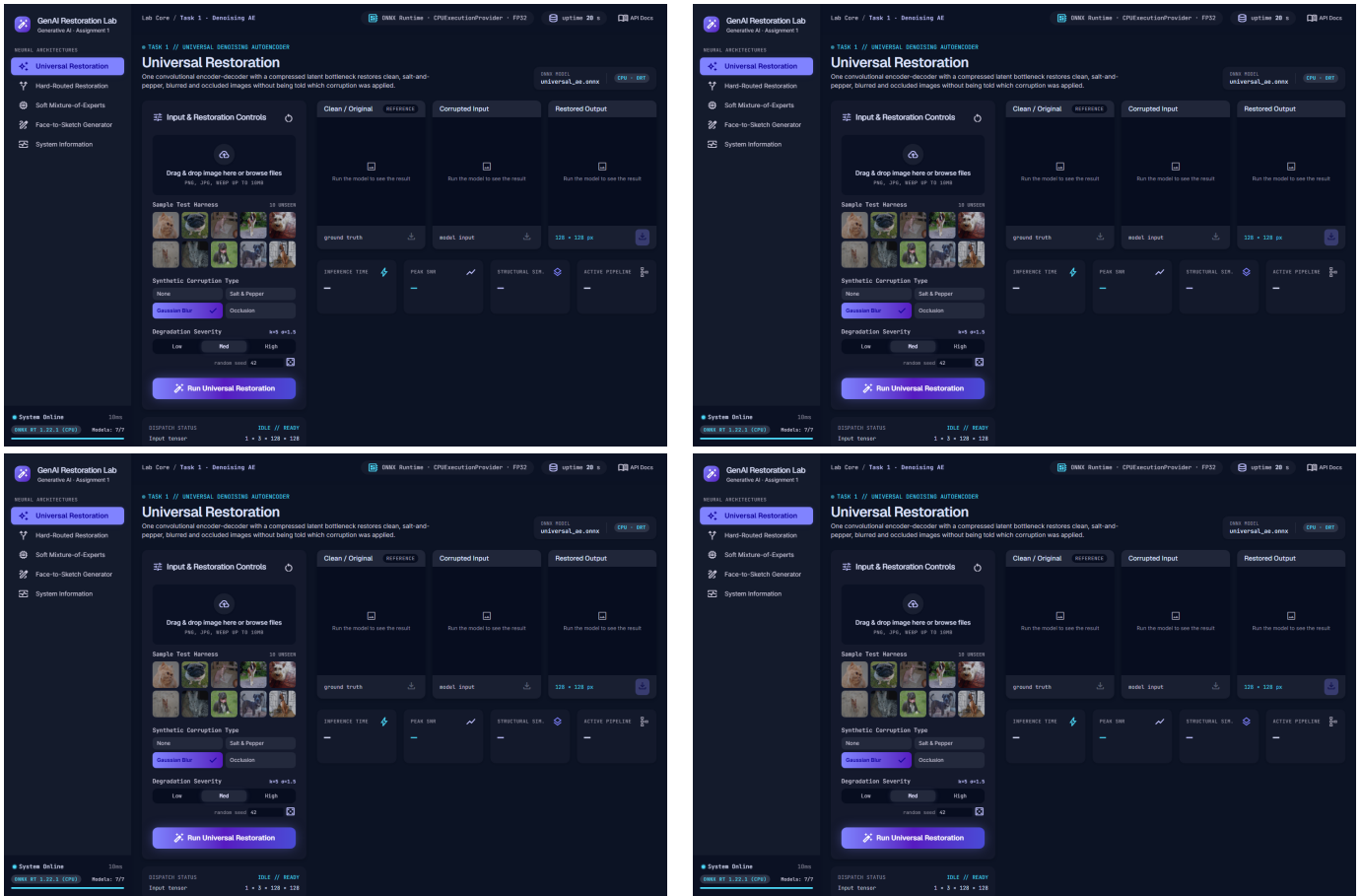


Fig. 5. The deployed application (running in Docker) on unseen test images: Universal Restoration, Hard-Routed Restoration, Soft Mixture-of-Experts Restoration and Face-to-Sketch Generator.

- [4] O. Ronneberger, P. Fischer and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in Proc. MIC-CAI, 2015, pp. 234–241.
- [5] H. Zhao, O. Gallo, I. Frosio and J. Kautz, “Loss functions for image restoration with neural networks,” IEEE Trans. Comput. Imaging, vol. 3, no. 1, pp. 47–57, 2017.
- [6] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, 2004.
- [7] B. Li, X. Liu, P. Hu, Z. Wu, J. Lv and X. Peng, “All-in-one image restoration for unknown corruption,” in Proc. CVPR, 2022, pp. 17452–17462.
- [8] V. Potlapalli, S. W. Zamir, S. Khan and F. S. Khan, “PromptIR: Prompting for all-in-one blind image restoration,” in Proc. NeurIPS, 2023.
- [9] R. A. Jacobs, M. I. Jordan, S. J. Nowlan and G. E. Hinton, “Adaptive mixtures of local experts,” Neural Comput., vol. 3, no. 1, pp. 79–87, 1991.
- [10] N. Shazeer et al., “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in Proc. ICLR, 2017.
- [11] W. Fedus, B. Zoph and N. Shazeer, “Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity,” J. Mach. Learn. Res., vol. 23, no. 120, pp. 1–39, 2022.
- [12] G. Hinton, O. Vinyals and J. Dean, “Distilling the knowledge in a neural network,” arXiv:1503.02531, 2015.
- [13] I. Goodfellow et al., “Generative adversarial nets,” in Proc. NeurIPS, 2014.
- [14] M. Mirza and S. Osindero, “Conditional generative adversarial nets,” arXiv:1411.1784, 2014.
- [15] P. Isola, J.-Y. Zhu, T. Zhou and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in Proc. CVPR, 2017, pp. 1125–1134.
- [16] A. Odena, C. Olah and J. Shlens, “Conditional image synthesis with auxiliary classifier GANs,” in Proc. ICML, 2017.
- [17] D.-P. Fan et al., “Facial-sketch synthesis: A new challenge,” Mach. Intell. Res., vol. 19, pp. 257–287, 2022.
- [18] O. M. Parkhi, A. Vedaldi, A. Zisserman and C. V. Jawahar, “Cats and dogs,” in Proc. CVPR, 2012.
- [19] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in Proc. KDD, 2019.
- [20] J. Bergstra, R. Bardenet, Y. Bengio and B. Kégl, “Algorithms for hyper-parameter optimization,” in Proc. NeurIPS, 2011.
- [21] A. Odena, V. Dumoulin and C. Olah, “Deconvolution and checkerboard artifacts,” Distill, 2016.
- [22] ONNX Community, “Open Neural Network Exchange,” <https://onnx.ai>, accessed 2026.

Appendix A Use of AI Tools

Tool. Claude Code (Anthropic, Claude Opus model) was used as a coding assistant inside VS Code. Google Stitch was used for the interface design, as required by the assignment.

Tasks it was used for. (i) Reading the assignment and planning the work under a tight schedule; (ii) writing the

first version of the training package (data pipeline, corruption functions, models, training loops, Optuna objectives, evaluation and ONNX export), the FastAPI back end, the React front end, the Dockerfiles and the Compose file; (iii) translating the Stitch export into React/Tailwind components; (iv) drafting the structure and the methodological text of this report, including the TikZ diagrams; (v) looking up dataset details (FS2K folder structure, annotation format and official split script).

How the outputs were tested and corrected.

- End-to-end smoke test: before using GPU time, the complete pipeline (Optuna, training, evaluation, figures, ONNX export) was run on CPU with synthetic data and a “smoke” budget. This exposed a crash when the batch size exceeded the number of samples (empty epochs and a division by zero), which was fixed in the batch sampler and data loader.
- Corruption checks: corrupted test images were inspected visually for all three severities (Section III), and the occlusion coverage was measured over 50 placements per severity (10.0 %, 20.0 %, 35.0 % ± 0.1 %) and over 2,000 training samples (10.9–34.2 %).
- Data pairing: FS2K photograph/sketch pairs were plotted side by side to confirm that the loader pairs files correctly; the stratified split was verified by printing the style counts.
- ONNX: every exported model is compared numerically with PyTorch (Table III).
- API: every endpoint was called with valid inputs, with a non-image file (rejected with HTTP 415) and with a path-traversal sample name (rejected with HTTP 404).
- Interface: headless-browser screenshots were compared with the Stitch renderings; invented hardware details from the Stitch mock-up were replaced by real values from the back end.
- Numbers in the report are not typed by hand: `scripts/make_report_assets.py` generates them from the result files written by the evaluation scripts.

All hyper-parameter values reported here were selected by Optuna, not by the assistant. I reviewed the generated code and can explain and modify every component.